

TECHNICAL DEEP DIVE

把缓存变成服务边界：Kimi-K3 在 vLLM 上的生产推理设计

从 Prefill/Decode 分离、分布式 KV，到多模态四级缓存、KDA checkpoint 与 EAGLE3 前缀复用

基于技术演讲与配套材料整理

配套材料：Kimi-K3 Production Serving.pptx

2026-08-29

原视频：[vLLM 小课堂（十五）：基于 vLLM 的 Kimi K3 智能体生产级推理服务](#) ·

配套资料：[Kimi-K3 Production Serving 课件](#)

Kimi-K3 的生产推理难题，不止是把 2.8T 参数的模型放进 GPU。权重就位后，多轮对话、长上下文、图片、工具结果和持续生成还会产生大量运行时状态。状态留不住，就要重复计算；计算阶段不分工，首 token 延迟与逐 token 延迟又会相互牵制。

本文沿真实因果链展开：先解释模型与 Agent 负载为何共同制造容量和时延压力，再分析 Prefill/Decode 分离、Mooncake 分布式 KV、图片四级缓存、Prefill 并行、KDA checkpoint、EAGLE3 no-drop，最后回到 Decode 的并行与混合量化。

文中的性能数字均来自演讲材料。凡缺少硬件、负载、统计口径或基线的结果，都会保留其适用边界，不将局部收益外推为端到端结论。

适读人群与前置知识

本文面向具备大模型推理、GPU 并行或分布式系统基础，但尚不了解 Kimi-K3 生产服务实现的工程师。

建议读者已经了解：

- Transformer 推理中的 Prefill、Decode 与 KV cache；
- 张量并行、专家并行和数据并行的基本目的；
- MoE、前缀缓存与推测解码的基本概念；
- TTFT、TPOT、吞吐和缓存命中率等指标的区别。

阅读目标

读完后，你应该能够：

- 解释 Kimi-K3 的模型结构与 Agent 负载为何共同放大状态压力；
- 理解 Prefill/Decode 分离和共享 KV pool 的架构动机；
- 区分图片四级缓存分别消除了哪一段重复工作；
- 理解 KDA checkpoint 与 EAGLE3 no-drop 处理的粒度错位；
- 正确解读材料中的 coverage、TTFT、接受率与量化收益。

一、问题不是模型能否运行，而是状态能否留得住

Kimi-K3 的生产难点不只在于把权重装进 GPU。权重占据大量显存后，多轮对话、长前缀和图像输入仍会不断产生运行时状态；与此同时，用户又要求首 token 尽快返回，后续生成保持流畅。容量、状态管理和时延因此不能被拆成三个孤立问题。

先看模型结构图，是为了确定权重之外还有哪些状态与计算路径必须被服务系统承载。

Kimi-K3 Model Structure

background

1 Hybrid attention

KDA linear attention mixed with MLA attention layers.

2 LatentMoE

896 routed experts, 16 active per token;

3 2.8T params

2.8T total parameters in MXFP4 — weights alone need 1.6 TB of GPU memory; on GB300 (288 GB HBM) that means at least 8 GPUs to serve one replica.

4 Multimodal + long context

Images support via the ViT; 1M-token context; prefix caching is a first-class problem.

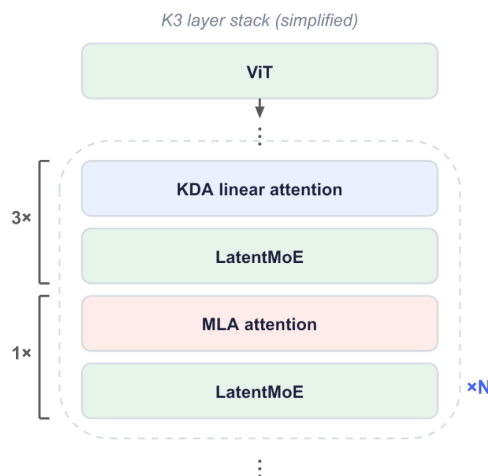


图 1：Kimi-K3 简化层栈及模型规模约束。来源：演讲 PPT，第 2 页。

图中的图像输入先经过 ViT (Vision Transformer, 视觉 Transformer)，转换为模型可以处理的视觉表示，然后进入重复的主干层组。虚线框交替包含三类关键模块：

- KDA：与 MLA 混合使用的一类线性注意力，其 recurrent state 是后续前缀缓存优化的对象；
- MLA：混合注意力中的另一类注意力；
- LatentMoE：按 token 路由到部分专家的稀疏混合专家结构。

左侧的 3× 与 1× 表示图示中的 KDA、MLA 组合比例，整个层组以 ×N 重复。页面没有给出 N，不能据此补全总层数。

LatentMoE 包含 896 个 routed experts，每个 token 激活其中 16 个。稀疏激活减少了单个 token 实际经过的专家数，但没有消除全部专家权重的部署需求。

页面将模型标为 2.8T 参数、总体权重格式为 MXFP4，并称权重本身需要 1.6 TB GPU 内存。在每卡 288 GB HBM 的 GB300 上，页面给出的结论是单副本至少需要 8 张 GPU。这里不能用 1.6 TB ÷ 288 GB 简单证明“8 张”：材料没有给出单位换算、并行粒度和显存预留的完整推导。

更重要的是，1.6 TB 只对应页面所述的权重规模，不包含 KV cache、激活、通信缓冲区和其他运行时开销。模型还支持图像输入与最长 1M-token context。1M 是能力边界，不表示生产请求都会达到这一长度；但它使长前缀是否能够持续保留，成为不能绕开的系统问题。

Prefix cache (前缀缓存) 通过复用已匹配前缀的推理状态，避免对相同历史重新执行 Prefill。Prefill 是处理输入上下文并建立后续生成所需状态的阶段；Decode 则基于这些状态逐 token 生成结果。

接下来需要看负载图，因为模型规模只决定容量底座，真实请求才决定状态增长速度和调度压力。

What production actually demands

Workload shape and SLA goals define the serving envelope

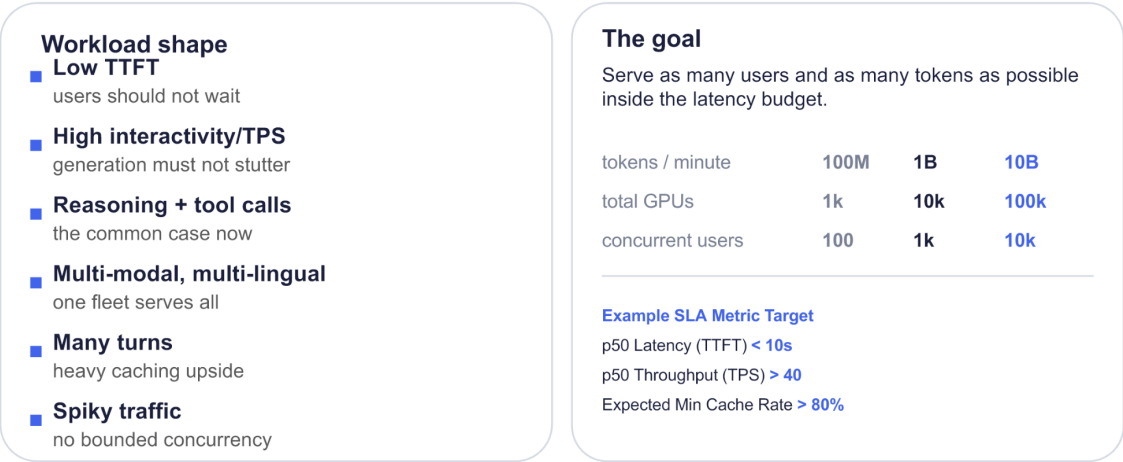


图 2：生产服务的负载形态、扩展目标与示例指标。来源：演讲 PPT，第 3 页。

模型结构与生产负载之间的对应关系如下。

生产特征	直接约束	对系统的影响
多轮会话	相同历史被反复引用	缓存价值上升，但驻留状态持续增长
推理与工具调用	工具结果被追加到上下文	前缀变长；状态丢失时重复 Prefill 增加
多模态输入	文字和图像进入同一请求	增加下载、预处理、视觉编码和 Prefill 路径
低 TTFT	首 token 不能等待太久	长 Prefill 与排队时间必须受控
高生成交互性	输出过程需要保持流畅	Decode 不能长期被大批输入计算阻塞
突发流量	请求到达存在峰值	调度和缓存容量需要承受短时冲击
工具格式要求	调用需遵循参数与 schema	服务目标还包括格式正确性

TTFT (Time To First Token) 表示首 token 延迟；TPOT (Time Per Output Token) 表示相邻输出 token 的时间间隔。页面列出的 p50 TTFT < 10s、p50 TPS > 40 和 Cache Rate > 80% 均为示例 SLA 目标，不是已经实现的测试结果。材料也没有定义 TPS 是单请求还是聚合口径，或 Cache Rate 是按请求、token 还是前缀长度统计。

可以用一次长会话说明状态如何膨胀：

- 1 用户提交长文和图片，系统完成视觉处理与首次 Prefill；
- 2 模型发起工具调用，工具结果被追加到同一会话；
- 3 用户继续追问，模型基于更长的历史生成；
- 4 后续轮次再次引用原始图片和工具结果。

如果历史状态仍能命中，后续步骤主要处理新增内容；如果缓存因容量不足被逐出，相同历史就可能再次参与 Prefill。重复计算会拉长 TTFT，并与正在 Decode 的请求竞争资源。

这条链路是基于材料的合理推断，演讲没有给出这个具体例子的实测时延。

页面还以 100M、1B、10B tokens/minute，以及 1k/10k/100k GPU、100/1k/10k 并发用户展示规模阶梯。这些数字应视为规划或示意，不能根据十倍递增推断系统已经验证线性扩展。

生产目标由此发生变化：重点不是让某个孤立请求达到峰值速度，而是在延迟预算和缓存覆盖目标内，持续扩大可服务的会话与 token 数量。下一步需要回答两个问题：历史状态放在哪里，以及 Prefill 与 Decode 是否还应共享同一批计算资源。

二、先拆计算阶段，再重画 KV 的流向

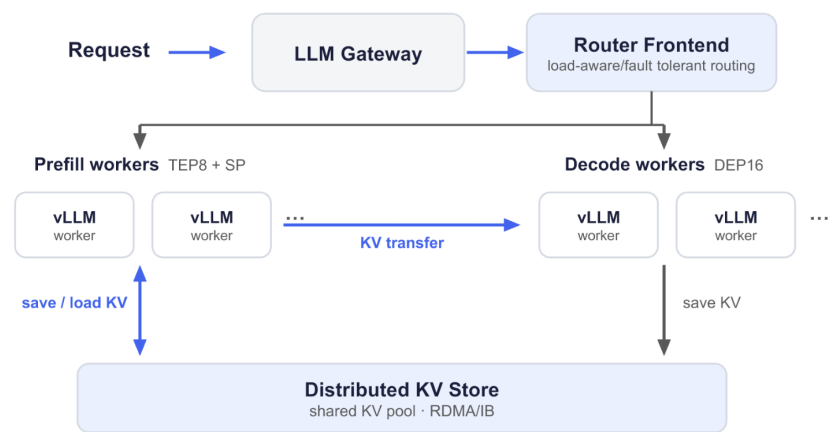
Prefill 与 Decode 处理的是同一次推理，却具有不同的资源目标。

Prefill 面向输入上下文，长请求会产生较大的集中计算；Decode 逐 token 推进，更关注 TPOT、batch 和 KV 容量。如果两者共享一组 worker，同一套资源配置就必须同时迁就长上下文计算和低延迟生成。

下面的系统图值得先看，因为它把请求流、计算流和 KV 状态流放在同一张架构图中，也解释了阶段分离后状态为何必须跨池移动。

Production Setup Overview

Multi-clusters GB300 platform — prefill and decode as separate pools



Prefill writes KV to the Mooncake pool; decode reads it back — prefix reuse across requests.

图 3：请求经网关与路由前端进入独立的 Prefill、Decode worker 池，KV 通过直接传输或共享存储跨池移动。来源：演讲 PPT，第 4 页。

图中的箭头需要分成三类理解。

第一类是请求流。请求从 LLM Gateway 进入 Router Frontend。材料将 Router 描述为负载感知且具备容错能力：它根据后端状态选择 worker，并处理节点故障带来的路由问题。

第二类是计算流。Router 后方是彼此独立的 Prefill worker 池和 Decode worker 池。Prefill 负责处理上下文并产生 KV，Decode 取得这些状态后继续生成。

页面分别标注了 Prefill 的 TEP8 + SP 与 Decode 的 DEP16。该页没有展开这些缩写，不能仅凭图示推导具体并行维度或 GPU 数量；后续页面给出了更明确的 Prefill、Decode 配置，应以后续材料为准。

第三类是 KV 状态流：

- 直接 KV transfer：把本次请求刚生成的状态从 Prefill 交给 Decode；
- 分布式 KV Store：Prefill 将 KV 写入共享池，Decode 从中读回，也可复用已经存在的前缀状态。

图中 Decode 一侧的箭头标成了 save KV，但页脚说明 Prefill 写入 Mooncake pool、Decode 读回。本文采用页脚表达的语义，并保留这一图文表面差异。材料没有说明直接传输与共享存储是并行执行、互为替代，还是按请求选择。

两类约束对应两个设计决定

生产约束	架构结果
并发会话持续积累历史 token，GPU HBM 无法保存全部 KV	将 KV 扩展到 GPU 之外，并用多机内存组成共享缓存池
TTFT、TPOT 限制聚合部署可接受的 batch	当同池部署无法同时满足效率和生成时延时，拆分 Prefill 与 Decode

并发会话越多、上下文越长，需要保存的 KV 就越多。材料没有完整给出单会话 token 分布、KV 大小和容量公式，因此无法给出通用阈值；能够确认的是，容量压力促成了 KV cache offloading，并进一步导向分布式 KV Store。

时延约束则推动了阶段分离。演讲介绍了一次早期实验：在 Prefill 与 Decode 尚未分离、同时采用讲者认为适合低延迟的技术时，为满足约 TPOT < 25 ms，最大 batch size 约为 4。

这个实验可以简化为以下状态：

- 四个请求共同进入聚合 worker；
- 继续扩大 batch 可能越过目标 TPOT；
- Decode 不能再通过增加 batch 摊薄开销；
- 同池中的 Prefill 也被这一较小 batch 约束。

这个结果只说明，在该模型、硬件和负载条件未完整披露的实验里，低延迟目标限制了聚合部署依靠扩大 batch 提升效率的空间。它不能用于计算 GPU 利用率，也不是 vLLM 或其他模型的通用阈值。

分离之后，Prefill 池可以围绕上下文计算配置资源，Decode 池则围绕逐 token 时延与 KV 容量组织 batch。相应代价是，KV 不再天然附着于同一个本地实例：系统必须解决状态的持久化、定位和传输问题。

三、Mooncake：把节点内存组织成共享会话池

阶段分离之后，历史 KV 如果仍然只保存在某个计算实例上，请求就会被缓存位置绑住。Mooncake Store 的作用，是把多个节点的主机内存组织成可跨实例访问的共享 KV pool。

先看 coverage 图，因为它直接展示了缓存边界如何从 GPU 扩展到单机内存，再扩展到跨节点共享池。

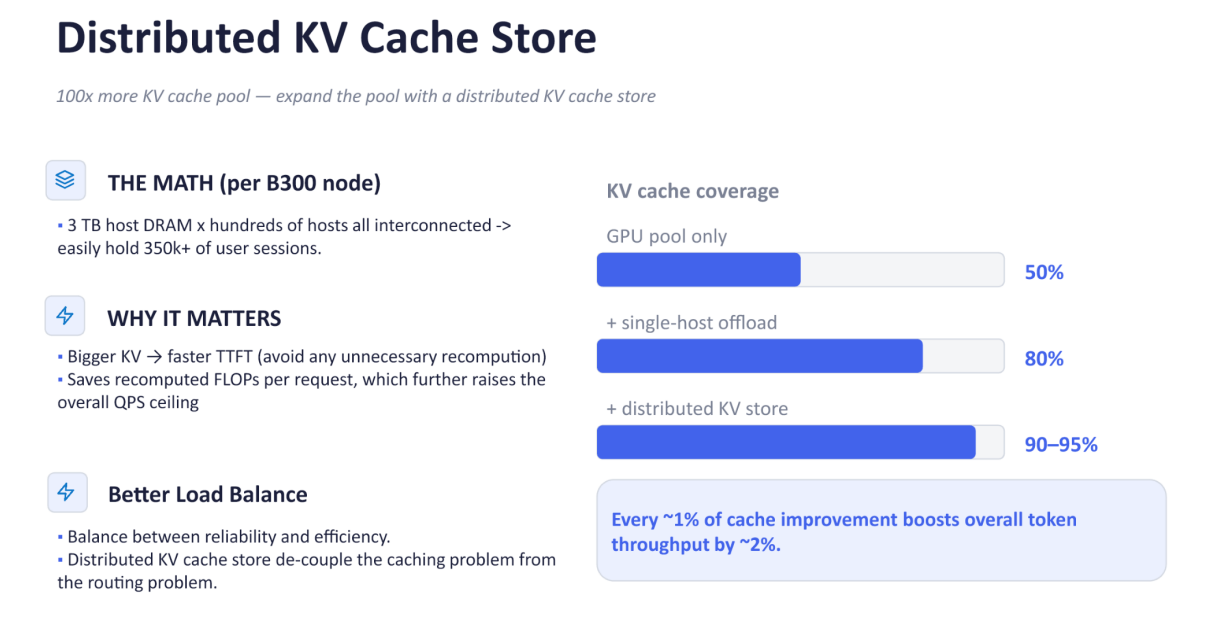


图 4：GPU pool、单机卸载与分布式 KV Store 的缓存覆盖率对比。来源：演讲 PPT，第 6 页。

三条进度条对应三层容量边界：

缓存层级	页面报告的 coverage	容量边界
仅 GPU pool	约 50%	受 GPU 显存容量约束
增加 single-host offload	约 80%	KV 可卸载到当前节点主机内存
增加 distributed KV store	约 90% - 95%	多节点主机内存形成共享池

材料没有给出 coverage 的精确定义，也没有披露工作负载、统计窗口和误差范围。50%、80% 和 90% - 95% 因而只能作为页面条件下的报告结果，不能视为任意集群都能达到的命中率。

容量边界从实例扩展到集群

页面称分布式存储可将 KV cache pool 扩大约 100 倍，并以 B300 节点为例估算：每个节点约有 3 TB 主机 DRAM，数百台主机互联后可容纳 350k+ 用户会话。

这里有三项限制：

- “100 倍” 的比较基线没有定义；
- 350k+ 是集群条件下的容量估算；
- 材料没有给出单会话 KV 大小、上下文长度、缓存精度、复制策略和元数据开销。

因此，不能从 3 TB 自行复算 350k+ 会话。这个数字支持的结论是扩展方向：KV 从局部 GPU 或单机资源，变为整个存储集群可以共同提供的资源。

演讲给出的 Mooncake 部署示例如下：

部署元素	示例配置或职责
计算规模	两个 GB300 NVL72 机架
控制平面	2 个 Mooncake Master 副本、3 个 etcd 副本
节点内 Store	按 NUMA 展示两个约 380 GiB RAM 的 store
数据通路	KV 经 RDMA 在节点间移动
vLLM engine	可独立升级、扩缩容或重启

Mooncake Store 与 vLLM engine 的生命周期分离后，已经卸载到独立 Store 实例的 KV 不会因为某个计算实例退出而同时丢失。不过，材料没有给出复制策略、故障注入结果、恢复时间、RDMA 延迟或跨机架成本，不能据此推断存储故障风险已经消失。

KV 命中如何影响 TTFT 与 QPS

第一条收益链发生在 Prefill：

命中历史 KV → 跳过对应前缀重算 → TTFT 缩短

例如，一个长会话被路由到新的 vLLM 实例。只要该实例能从 Mooncake Store 读回匹配的历史 KV，就不必因为本地没有缓存而重新处理完整历史。

省下的计算又形成第二条链：

避免 Prefill 重算 → 节省 FLOPs → 计算资源接入更多请求 → QPS 上限提高

页面报告了一条经验关系：缓存改善约 1%，整体 token throughput 约提高 2%。材料没有提供适用区间、误差或因果分解，所以不能把它外推为普适线性公式。

问答中，讲者还给出一类不含图片的文字会话示例：约 20~30 轮以上，每轮输出约 300~600 token、输入约 2k~4k token，总长度约 100k~200k token；在这一范围内，约 90% 被认为是较客观的缓存目标。该例有助于理解长会话为什么值得保留，但不一定与图中的 90% - 95% coverage 使用相同统计口径。

共享状态如何弱化路由冲突

没有共享存储时，Router 往往同时追求两个目标：

- 将请求发往负载较低的实例；

- 将请求发回保存其历史 KV 的实例。
两者可能冲突。持有缓存的实例可能已经较忙，而空闲实例没有相应状态，需要重新 Prefill。
共享 KV pool 将关系改写为：

KV 可跨实例读取 → 请求不必仅为本地命中而粘在某个实例 → Router 更专注于实例负载

这里的“解耦”不是说 Router 可以完全忽略缓存位置和传输代价，而是前缀复用不再严格依赖原来的计算实例。远端读取是否比重算更划算，仍取决于前缀长度、网络成本和并发流量。

讲者称，在其观察到的特定生产流量下，分布式 KV 与 Prefill/Decode 分离的流量尚未使 IB 带宽成为主要瓶颈；其解释之一是请求大部分时间处于 Decode，并发上限通常更早受到 KV pool 容量限制。这只是特定部署下的经验观察，材料没有提供网络拓扑、带宽利用率或压力测试数据。

Mooncake 扩展的是已经生成的模型状态。图片在进入 LLM Prefill 之前，还会经历下载、预处理和视觉编码；仅有分布式 KV，并不能消除这段重复链路。

四、同一张图片为什么需要四层缓存

多轮 Agent 请求会重复携带相同图片 URL。单个请求最高约出现 150 张图片，这是上限式描述，不表示每轮固定包含 150 张，也不表示这些图片彼此不同。

一次完全未命中的图片会经历：

URL → HTTP fetch → raw bytes → / /  → pixel tensor → ViT embedding →  prompt → LLM Prefill → prefix KV

这条链路包含四种不同的数据产物：原始字节、像素张量、视觉 embedding 和模型 KV。它们位于不同的计算边界，因此不能由一个缓存层统一替代。

下面这张图值得看，是因为它明确标出了每层缓存保存什么，以及一次命中究竟跳过哪段工作。

Optimization — Media Cache

Image-heavy traffic; multi-turn chats repeat the same images — fetch pressure was huge

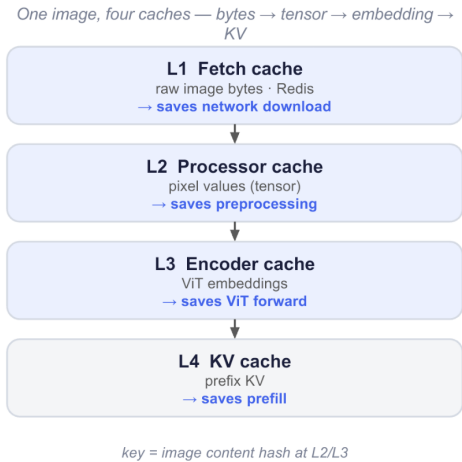
- 1

L1 · Fetch cache
Raw image bytes, served from a standalone distributed **Redis** server — saves the HTTP download. Hit rate **95%**.
- 2

L2 · Processor cache
MultiModalProcessorCache — key: image content hash, value: preprocessed tensors (resized / normalized pixel values).
- 3

L3 · Encoder cache
Key: image hash, value: ViT output **embeddings** — a hit skips the whole ViT forward.
- 4

L4 · KV cache
The same **prefix cache** as text-only requests — the last layer in the chain.



~53% TTFT saved (network RTT + remote fetch + preproc + ViT + LM prefill).

图 5：L1 至 L4 依次缓存 bytes、tensor、embedding 和 KV。来源：演讲 PPT，第 9 页。

图中的箭头表示数据形态逐步转换。命中位置越靠后，能够跳过的处理阶段越多。

层级	缓存值	材料明确的键	命中后消除的工作
L1 Fetch cache	原始图片字节	未说明完整键结构	HTTP 下载及相关等待
L2 Processor cache	预处理后的 pixel tensor	图片内容哈希	解码、缩放和归一化
L3 Encoder cache	ViT embedding	图片哈希	ViT forward
L4 Prefix KV cache	prefix KV	沿用文本前缀缓存机制，完整键结构未说明	已匹配前缀的 LLM Prefill

L1 部署在独立的分布式 Redis 中。命中后，系统仍需执行图片预处理、ViT 编码和 LLM Prefill，但远端下载已经退出关键路径。

L2 使用 MultiModalProcessorCache，以图片内容哈希缓存预处理后的 tensor。它保护的是图片解码与预处理路径。

L3 保存 ViT embedding。命中后，可以跳过视觉编码器前向。演讲时，Encoder cache 仍被描述为不够完善并处于内部生产实验阶段，因此不能将其写成已经全面成熟上线的能力。

L4 保存 prefix KV。当图片 embedding 与文本 prompt 形成的完整前缀满足匹配条件时，系统可以复用相应模型状态，跳过该部分 LLM Prefill。它沿用文本请求的前缀缓存机制，并非单独定义一套图片 KV。

以同一张图片 A 的两轮请求为例。第一轮四层均未命中：

- 1 下载 A，将 raw bytes 写入 L1；
- 2 解码、缩放和归一化，将 tensor 写入 L2；

- 3 执行 ViT forward，将 embedding 写入 L3；
- 4 合并 prompt 并执行 Prefill，将 prefix KV 写入 L4。

第二轮再次携带 A 时，系统从能够命中的最深层结果继续：

- 只有 L1 命中：仍需预处理、ViT 和 Prefill；
- L2 命中：从 pixel tensor 继续；
- L3 命中：直接取得视觉 embedding；
- L4 也命中：对应的 LLM Prefill 同样可以跳过。

因此，四层缓存不是重复保存同一数据，而是分别保护网络、图像预处理、视觉编码器和语言模型四类资源。

页面报告 L1 Fetch cache 命中率为 95%，这一数字不能用于推断 L2、L3 或 L4 的命中率。页面还报告四层链路合计约节省 53% TTFT；演讲以约 20 秒降至约 10 秒作为近似示例。

材料没有给出硬件、样本规模、基线定义和延迟分位数，也没有将收益分摊到各层。因此，53% 只能作为整条链路的合计报告，不能用于预测任意业务流量，更不能写成某一缓存层的独立收益。

四层媒体缓存把复用边界推进到了 Prefill 之前。不过，新增文本、前缀变化和缓存未命中仍会触发 Prefill。下一步要处理的是：剩余计算如何分片执行，以及 KDA 状态为什么仍会受块边界约束。

五、Prefill 的并行骨架：计算为何保持分片

Kimi-K3 的 Prefill 同时包含混合注意力与 MoE 计算。系统需要避免在注意力结束后过早让所有 rank 都恢复并持有完整 token 表示，否则中间 residual 与 MoE 路径可能增加重复计算和通信。

生产配置采用 TP8+EP8，并启用 Sequence Parallelism：

机制	分片对象	Prefill 中的职责
Tensor Parallelism (TP)	算子的张量维度	以 TP8 处理 hybrid attention
Expert Parallelism (EP)	MoE experts	以 EP8 将专家分散到不同 rank
Sequence Parallelism (SP)	token 序列	让中间路径中的 rank 只持有自身 token shard

材料没有给出 TP8 与 EP8 的具体进程组映射，因此不能把两者相乘为 64 个 rank，也不能断言它们复用同一个 8-rank group。

下面的执行图需要重点看，因为它展示了完整表示如何经 reduce-scatter 变成 token shard，又如何在下一层前恢复。

Deployment Strategy——Prefill

Prefill side: parallelism and backend choices

- 1 Parallelism: TP8+EP8**
TP for hybrid attention ; EP shards MoE experts across ranks.
- 2 Attention backends**
MLA → **TOKENSPEED_MLA**; KDA → **FlashKDA**.
- 3 MoE backend: MegaMoE**
MegaMoE (DeepGEMM) for the expert GEMMs.
- 4 Sequence parallelism on**
SP shards tokens across ranks; pairs with reduce-scatter / all-gather.

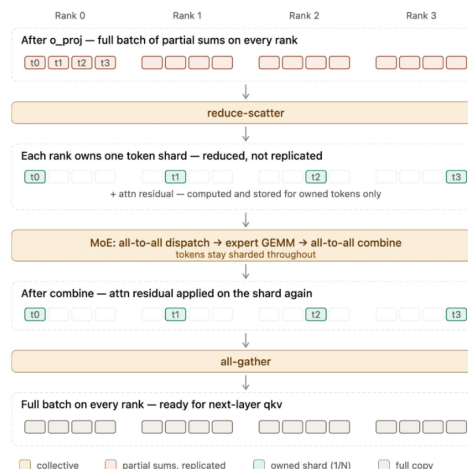


图 6：Prefill 侧的并行配置、计算后端及 Sequence Parallelism 数据流。来源：演讲 PPT，第 10 页。

图中使用 Rank 0 至 Rank 3 解释流程。这只是教学示意，生产配置仍是 TP8+EP8。

- 完整表示与 partial sums** 进入该层时，各 rank 面向完整 token 集合执行张量并行计算。注意力 **o_proj** 后，每个 rank 持有局部计算产生的 partial sums，而不是最终完整输出。
- reduce-scatter** partial sums 经 reduce-scatter 完成求和归约，并沿 token 维度分发。每个 token 的最终结果只落到所属 rank。
- 初始 token shard** reduce-scatter 后，每个 SP rank 只保留全局 token 的一部分。此时的 token 归属描述的是进入 MoE dispatch 前的初始分片，不等同于该 rank 随后实际执行 expert GEMM 的全部输入。
- MoE dispatch** token 通过 all-to-all dispatch 发往持有目标专家的 rank。EP 决定专家位于哪里，SP 决定 dispatch 前各 rank 持有哪些 token。完成 all-to-all 后，一个专家 rank 可能接收来自多个 SP rank 的 token。
- expert GEMM 与 combine** 持有专家的 rank 对所有路由到本地专家的 token 执行矩阵乘，其数量不再受初始 token shard 大小限制。随后，all-to-all combine 将结果送回各 token 原来的归属位置。
- 最终 all-gather** MoE 结果与 residual 合并后，系统执行 all-gather，恢复完整 token 表示，供下一层 QKV 计算使用。

用 4 个示意 rank、8 个 token 可以得到最小状态演进：

Rank	reduce-scatter 后、dispatch 前初始持有的 token
Rank 0	t_0,t_1
Rank 1	t_2,t_3

Rank	reduce-scatter 后、dispatch 前初始持有的 token
Rank 2	t_4,t_5
Rank 3	t_6,t_7

此时每个 SP rank 初始拥有两个 token。进入 all-to-all dispatch 后，持有专家的 rank 会处理来自不同 rank、被路由到本地专家的 token，实际数量不再限定为两个。expert GEMM 完成后，combine 将结果送回原 token 的归属位置；直到下一层 QKV 之前，all-gather 才恢复 t_0 至 t_7 的完整表示。

材料还给出了对应计算后端：

- MLA 使用 `TOKENSPEED_MLA`;
- KDA 使用 `FlashKDA`;
- 专家 GEMM 使用基于 DeepGEMM 的 `MegaMoE`。

材料没有提供这些后端或 TP8+EP8+SP 相对其他配置的吞吐、TTFT、显存占用和扩展效率数据。因此，本节只能确认执行结构，不能将其直接写成已量化的性能收益。

这套并行骨架解决了 Prefill 计算如何分摊，却没有解决状态能否在任意位置复用。KDA recurrent state 仍受前缀块边界约束，可能让同一批 token 因状态对齐被重复执行。

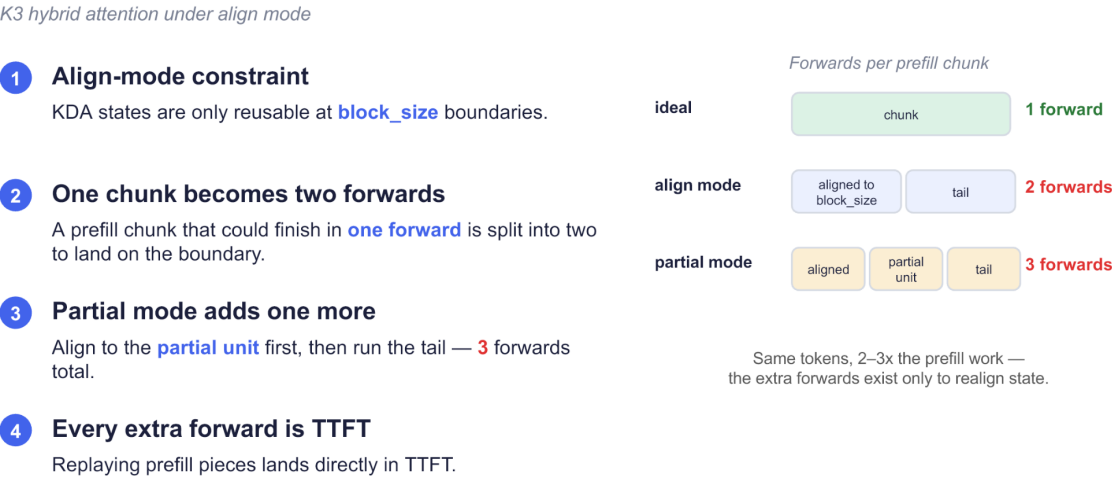
六、KDA checkpoint：在一次前向中截取可复用状态

KDA 需要缓存随前缀递推形成的 recurrent state。旧路径只能在预设边界取得可复用状态，因此为了让状态落在 block 或 partial-unit 边界上，需要通过额外 forward 重放相关 token。页面将这种工作放大概括为：align mode 需要两次 forward，partial mode 需要三次，对应约 2~3 倍 Prefill work。

Checkpoint mode 的目标，是让 FlashKDA 在一次完整 forward 中直接导出指定位置的 recurrent state，从而不再为了取得边界状态而重复执行相同 token。

先看旧路径，是为了区分“输入序列变长”和“同一 token 因状态对齐被重放”这两种情况。

Kimi K3 Prefix Cache — The Problem



Optimization — Checkpoint Mode

Export KDA state at the position you actually need

- 1 Kernel-side export**
FlashKDA exports the recurrent state at the requested checkpoint position, mid-chunk.
- 2 Per-request checkpoint block**
Each request gets an **extra checkpoint block**; shared prefix blocks are never overwritten.
- 3 Fine granularity**
128-token **partial unit** checkpoints inside the **1536**-token block— no replay from the previous page boundary.
- 4 One forward, done**
No chunk splitting: a single forward. Speed up TTFT P50 **-40%** vs baseline.

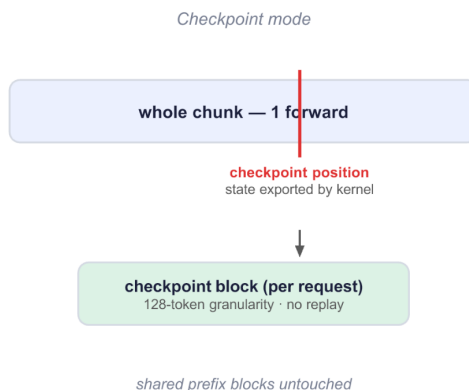


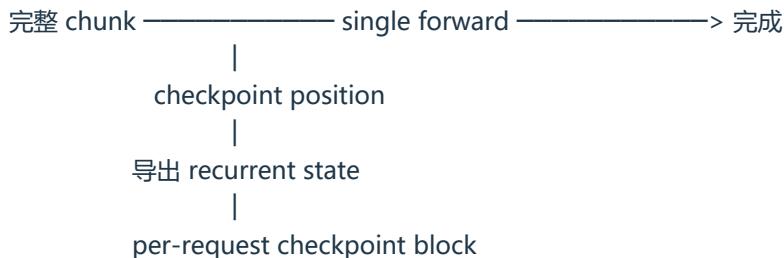
图 8：Checkpoint mode 的中间状态导出与请求私有 checkpoint block。来源：演讲 PPT，第 12 页。

图中的水平长条表示整个 chunk，竖线表示 **checkpoint position**。修改后的 FlashKDA kernel 计算到该位置时导出 recurrent state，但 forward 不结束，而是继续处理后面的 token。

状态被写入每个请求独立维护的 **checkpoint block**。这样做有两个作用：

- 请求中途产生的状态不会覆盖共享 prefix blocks；
- 状态不再需要通过额外 forward 和 token 重放取得。

新路径可以概括为：



因果关系是：

FlashKDA 支持中途导出状态 → 边界状态不再依赖额外 forward → 无需为对齐重放相同 token → 每个 chunk 恢复为一次 forward

页面给出的 block 大小为 1536 token，partial unit 为 128 token。两者构成 12 个粒度单位，但这只能说明粒度关系，不能据此断言实现一定保存 12 个独立 checkpoint。

页面报告 checkpoint mode 使 TTFT P50 相对未定义 baseline 降低 40%；演讲也将线上观察描述为相对先前实现约下降 40%。这个数字仅适用于 TTFT

P50。材料没有给出硬件、请求长度、并发、缓存命中率、样本数或等尾延迟，不能将其外推为所有负载的平均时延或吞吐收益。

P95、P99

方案也存在成本：每个请求需要额外 block，会增加内存占用，但材料没有量化其大小和总体开销。

checkpoint

KDA checkpoint 解决的是 KDA 状态导出粒度问题。然而，目标模型状态能够细粒度复用，并不代表推测解码的草稿 KV 也完全由前缀决定。EAGLE3 的最后一个草稿槽还依赖前缀结束后的 token，这会引出另一种粒度错位。

七、EAGLE3 的一位错位，为什么会牺牲整块缓存

EAGLE3 是使用目标模型隐藏状态生成草稿 token 的推测解码方案。它的草稿输入相对目标序列存在一位偏移，因此最后一个草稿槽并不完全由共享前缀决定。

先看依赖图，因为问题的核心不是缓存实现，而是最后一槽在语义上依赖哪个 token。

EAGLE3 × Prefix Cache — Drop the Last Block

EAGLE's draft input is left-shifted by one token

- Left-shifted draft input**
Draft slot i pairs target hidden h_i with the **next** token x_{i+1} , not x_i .
- The last slot looks past the prefix**
Slot $N-1$ needs x_N — sampled **after** the prefix ends, so the prefix hash does not cover it.
- Same prefix ≠ same last slot**
Two requests with an identical prefix can sample different x_N — a hit would reuse **stale draft KV** and drag down acceptance.
- Granularity forces a full block drop**
Ideal rollback is **1 token**, but cache hits are expressed in **128-token blocks** — so vLLM replays a whole block.

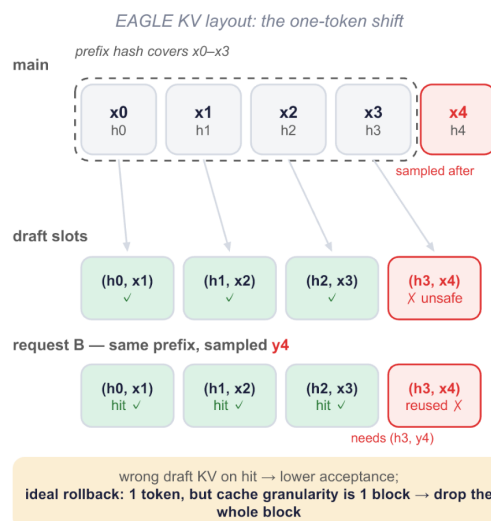


图 9：前三个草稿槽由前缀确定，最后一槽依赖前缀结束后的采样结果。来源：演讲 PPT，第 13 页。

设目标模型前缀为 $x_0, \dots, x_3$ ，对应隐藏状态为 $h_0, \dots, h_3$ 。草稿槽 i 使用 h_i 与下一个 token x_{i+1} ：

$$\text{draft_slot_i} = (h_i, x_{i+1})$$

对应关系如下：

草稿槽	是否由前缀确定	原因
(h_0,x_1)	是	两个输入均位于前缀内
(h_1,x_2)	是	两个输入均由前缀决定
(h_2,x_3)	是	两个输入均由前缀决定
(h_3,x_4)	否	x_4 在前缀结束后采样

考虑两个拥有相同前缀的请求。请求 A 在前缀后采样出 x_4，缓存中留下 (h_3,x_4)；请求 B 可能采样出 y_4，真正需要的是 (h_3,y_4)。

如果 B 直接复用 A 的末槽，陈旧的草稿 KV 可能降低 draft acceptance，也就是目标模型接受草稿 token 的比例。这里不能写成最终输出错误，因为目标模型仍会验证草稿；不匹配候选可以被拒绝。

从语义依赖看，理论上只需回滚 1 token。但 vLLM 的前缀缓存命中以 128-token block 为粒度，不能只将块内最后一个 token 标记为未命中：

末槽不确定 arrow 理想回滚 1 token arrow 实际粒度为 128-token block arrow 默认丢弃整个尾块

丢弃尾块不意味着完整前缀缓存都失效；被放弃的是包含不安全末槽的最后一个匹配块。

no-drop：更多复用与接受率风险之间的选择

disable_eagle_block_drop 是可选开关。启用后，vLLM 不再丢弃已经匹配的尾块，即采用 no-drop 策略。它增加了前缀复用机会，但也保留了末槽可能陈旧的风险。

下面的实验图需要结合测试范围阅读，因为它只回答 MT-Bench 两轮 A/B 中发生了什么，不能证明 no-drop 在所有负载下都合适。

Optimization — EAGLE3 No-Drop Prefix Cache

Reuse the trailing prefix block EAGLE normally discards

- 1

What

Opt-in switch `disable_eagle_block_drop` — keep the trailing matched prefix-cache block instead of dropping it as volatile.
- 2

Why

More prefix reuse for speculative decode; applies to local prefix cache, Mamba chunk alignment, KV offload, and Mooncake Store.
- 3

Result (MT-Bench 2-turn A/B)

turn-2 hits 9/80 → 36/80; draft acceptance neutral (−0.03pp).

MT-Bench two-turn A/B — default vs no-drop

Metric	drop	no-drop	Δ
Turn-2 cache hits	9/80	36/80	+27
Draft acceptance	59.81%	59.79%	−0.03pp
Accept length	2.794	2.794	−0.001

图 10：默认丢弃尾块与 no-drop 的 MT-Bench two-turn A/B 对照。来源：演讲 PPT，第 14 页。

PPT 给出的结果为：

指标	drop	no-drop	PPT 标注差值
第二轮 cache hits	9/80	36/80	+27
Draft acceptance	59.81%	59.79%	-0.03pp
Accept length	2.794	2.794	-0.001

第二轮 cache hits 从 9 次增至 36 次，即增加 27 次，不能写成增加 27 个百分点。

Draft acceptance 的显示值从 59.81% 变为 59.79%，差值按 PPT 原样记录为 -0.03pp。Accept length 两列均显示 2.794，表中差值却为 -0.001。这些表面差异可能来自显示舍入，不能根据可见数字自行修正原表结果。

这组 A/B 表明，在该限定场景中，no-drop 增加了尾块命中，同时接受率与接受长度变化较小。但材料没有给出硬件、随机性、请求长度分布或显著性检验，“变化较小”不能提升为统计等价。

因此，no-drop 是一项工作负载相关的取舍：

- 前缀重复较多、尾块重算成本较高时，更多复用可能值得；
- 续写分叉频繁时，陈旧末槽可能更明显地拖累接受率；
- 是否启用应依据实际接受率和缓存收益，而不是将其设为普适默认值。

前缀尾块问题处理后，Decode 的整体并发仍然受 KV 容量、注意力执行和逐 token 线性投影成本约束。

八、Decode 收口：容量优先的并行与混合量化

Decode 的首要约束并不只是单步计算速度，还包括能否为目标 batch 保留足够的历史状态。上下文越长、并发越高，KV cache 占用就越大。

页面将 Decode 描述为 capacity-bound，应理解为该部署首先受 KV 容量约束；这不意味着计算、显存带宽和跨卡通信在所有负载下都不会成为瓶颈。

先看 Decode 组件图，是为了将并行布局、注意力后端和推测解码放回同一执行路径。

Deployment Strategy — Decode

Decode is capacity-bound — every rank must hold KV

1 Parallelism: DP16+EP16

Attention **TP1+DP16** (no KV duplicated); **EP16** for the MoE

2 KV capacity bound

An 8-GPU deployment caps the achievable **batch size** — KV capacity is the limiter.

3 Attention backends

FlashInfer MLA + **fused decode KDA kernel**.

4 Speculative decoding

EAGLE 3.1 drafts a block from K3's own hidden states; K3 verifies it in one pass.



图 11：Decode 以 KV 容量为中心组织 DP16、EP16、注意力内核与 EAGLE 3.1。来源：演讲 PPT，第 15 页。

注意力采用 **TP1+DP16**：Tensor Parallelism 为 1，Data Parallelism（数据并行）为 16，并注明“不复制

KV”。这里能够确认的是页面对该布局的描述，不能将其扩大解释为整个系统不存在任何状态副本。

MoE 使用 **EP16**，即以 16 路 Expert Parallelism 组织专家计算。**DP16+EP16** 描述的是不同并行维度，不能相加成 32 张 GPU。

页面还同时出现 DP16 和“8-GPU deployment”，但没有解释两者所处的部署层级，必须保留这一不确定性。材料能够支持的结论是：Decode 侧会围绕 KV 容量比较并行配置，页面所述 8-GPU 部署的最大 batch 会受到 KV 容量限制。

注意力后端包括：

- MLA 使用 FlashInfer MLA；
- KDA 使用 fused decode KDA kernel。

页面只给出了最终采用的后端，没有提供候选内核之间的基准，因此不能判断它们分别贡献了多少吞吐或延迟收益。

EAGLE 3.1 则从 Kimi-K3 的隐藏状态生成草稿块，再由目标模型通过一次前向验证候选。最小执行过程是：

- 1 Kimi-K3 产生当前隐藏状态；
- 2 草稿侧生成候选 token；
- 3 Kimi-K3 一次前向验证候选序列；
- 4 接受连续匹配部分，在首次不匹配处停止。

其实际收益取决于草稿长度和接受率。材料没有给出固定草稿块长度、相对性能收益或额外显存开销。

单 checkpoint 下的两份量化配置

最后看量化图，是为了区分 checkpoint 的组织方式、融合 GEMM 的正确性结论和局部性能结果。

MXFP8 Attention Mixed Quantization

Serve the dual-config MXFP8-attention + MXFP4-experts checkpoint as-is

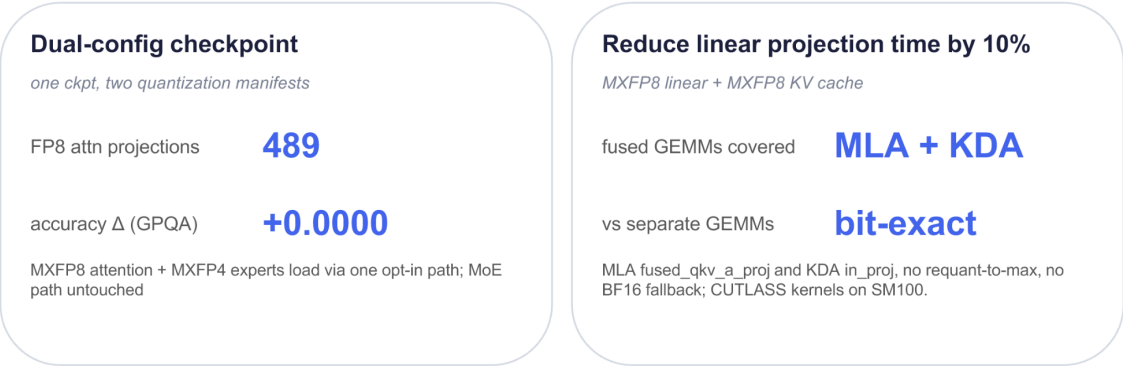


图 12：双配置 checkpoint 的加载方式、融合 GEMM 覆盖范围与局部指标。来源：演讲 PPT，第 16 页。

混合量化采用 “一个 checkpoint、两份 quantization manifest” 的组织方式。页面给出的配置包括：

- 489 个 FP8 attention projections；
- MXFP8 attention；
- MXFP4 experts；
- 可选的混合量化加载路径。

“MoE path untouched” 表示新增加载路径没有改动既有 MoE 计算路径，不代表 experts 没有量化。

页面报告 MXFP8 attention 覆盖 MLA、KDA 的融合 GEMM；对于这部分实现，材料只证明其相对 separate GEMMs 为 bit-exact。相关性能证据必须逐项限定：

指标	可以支持的结论	不能外推的结论
GPQA accuracy Δ +0.0000	页面显示精度下未观察到差值	数学意义上的零误差，或其他基准精度不变
bit-exact	融合 GEMM 相对 separate GEMMs 逐位一致	相对 BF16、FP16 原模型逐位一致

指标	可以支持的结论	不能外推的结论
线性投影时间降低 10%	SM100 上 CUTLASS kernels 的局部投影耗时下降	总 Decode 延迟或端到端吞吐提高 10%

线性投影的 10% 结果没有披露 batch、序列长度、基线时延、重复次数和误差范围，也不能与推测解码、缓存和并行布局的收益直接相加。

因此，现有材料可以确认三类设计：

- 以 KV 容量为核心选择 Decode 并行布局；
- 按逐 token 执行形态匹配 MLA、KDA 后端；
- 通过 EAGLE 3.1 和混合量化降低部分计算路径的成本。

现有证据不能支持完整端到端加速比例。

结论：优化的对象是状态的生命周期

Kimi-K3 生产推理中的各项设计共同围绕一个问题展开：已经付出计算得到的状态，能否在正确的时间、位置和粒度上继续复用。

- 2.8T 参数、长上下文、混合注意力、稀疏 MoE 与图片输入共同压缩运行时状态空间；多轮 Agent 请求又持续扩大需要保留的历史。
- Prefill/Decode 分离处理资源目标差异：Prefill 面向长上下文计算，Decode 面向逐 token 时延与 KV 容量。分离之后，KV 的移动和持久化成为显式系统职责。
- Mooncake Store 将 KV 从单个计算实例的生命周期中分离出来。共享状态既能减少前缀重算，也能弱化缓存命中与负载均衡之间的冲突。
- 四级媒体缓存把复用边界从 LLM KV 向前推进到图片字节、预处理 tensor 和 ViT embedding。只缓存最终 KV，无法消除图片进入 Prefill 之前的重复工作。
- KDA checkpoint 处理状态导出粒度：让 FlashKDA 在一次 forward 中间导出 recurrent state，避免为对齐 block 和 partial unit 重放相同 token。
- EAGLE3 no-drop 处理语义依赖与缓存块粒度的错位。它在限定 A/B 中增加了尾块命中，但保留了末槽陈旧可能降低草稿接受率的风险。
- Decode 的 DP/EP 布局、专用内核、推测解码和混合量化分别作用于容量或局部计算成本，不能把各自数字相加成端到端收益。

证据局限

本文材料仍缺少若干决定生产结论的重要数据：

- SLA、coverage 和缓存经验关系缺少统一统计口径；
- 多项时延结果缺少硬件、请求长度、并发、样本数和尾延迟；

- Mooncake 未披露远端读取、复制、跨机架传输与故障恢复成本；
- KDA checkpoint 的额外 per-request block 内存没有量化；
- EAGLE3 no-drop 只有 MT-Bench two-turn A/B，缺少更广泛负载和显著性信息；
- Decode 页面没有解释 DP16 与 8-GPU deployment 的层级关系；
- 量化的 10% 仅对应 SM100、CUTLASS kernels 下的线性投影时间；
- 材料没有提供能够汇总所有局部优化的端到端延迟、吞吐和资源成本对照。

在这些边界内，可以得到一套清晰的工程方法：先识别状态在哪个阶段产生，再决定它应当留在 GPU、本机内存、共享存储，还是更靠前的媒体处理层；随后按 Prefill 与 Decode 的不同瓶颈配置计算资源，并让缓存粒度尽可能贴合真实的语义依赖。